Street-View Digit Detection Based on DINO

Visual Recognition HW2

You-Zhe Xie

1. Introduction

The task is street-view digit detection: given an RGB image, the model must predict the bounding box and class label for every digit present. The dataset contains 30,062 training images, 3,340 validation images, and 13,068 test images annotated in COCO format. The assignment requires DETR as the main architecture with ResNet-50 as the backbone;

Although vanilla DETR successfully reformulates detection as set prediction — eliminating NMS and hand-crafted anchors — it has two practical drawbacks: (1) extremely slow convergence, typically requiring hundreds of epochs to reach acceptable performance; and (2) poor detection of small objects. I therefore re-implemented DETR as its successor DINO, incorporating the following designs:

- Multi-scale deformable attention replaces single-scale global attention, improving detecting different size digits and reducing complexity.
- Mixed query selection gives the decoder a reasonable spatial prior from the start, accelerating convergence.
- Contrastive denoising (CDN) stabilises Hungarian matching and further speeds up convergence.
- Look Forward Twice allows earlier decoder layers to benefit from the box-refinement information produced by later layers.

After iterative ablation experiments, the final model achieves a validation mAP (COCO metric, IoU=0.5:0.95) of approximately 0.4611.

2. Method

2.1 Model Architecture

The overall model consists of four stages: (1) a multi-scale ResNet-50 backbone, (2) a Transformer encoder, (3) two-stage query selection plus a Transformer decoder, and (4) per-layer classification and bounding-box prediction heads. Each component is described below in terms of the problem it addresses and the rationale for the chosen design.

Multi-Scale Backbone

Problem: When objects vary greatly in size, a single-scale feature map cannot simultaneously capture the semantic information of large objects and the fine-grained details of small ones. In practice, I found that detecting small digits is the hardest challenge in this task.

Approach: I use an ImageNet-pretrained ResNet-50 as the backbone and extract feature maps at three different scales to form a feature pyramid, allowing the downstream Transformer to see both high-level semantics and relatively fine spatial detail. Each scale is projected to a uniform dimension via a 1×1 convolution followed by GroupNorm, which avoids the instability of BatchNorm under small batch sizes (adding more scales significantly increases VRAM usage).

Multi-Scale Deformable Transformer Encoder

Problem: The global self-attention in vanilla DETR has $O(N^2)$ complexity with respect to the number of tokens. Introducing multi-scale features causes the token count to explode, making memory and computation intractable. Moreover, all spatial positions are treated equally, so the model takes a long time to learn where to attend.

Approach: I adopt multi-scale deformable attention: each query aggregates over a small set of learned 2-D sampling locations, focusing on important information and reducing complexity to near-linear in the number of tokens. The concrete architecture is a standard Encoder–Decoder Transformer (3 layers each) where the attention module is replaced with deformable attention.

Mixed Query Selection

Problem: In vanilla DETR, all object queries are learnable embeddings shared across all images: the decoder must search for object locations from scratch with no spatial prior. This is one of the core reasons DETR converges slowly.

Approach: A lightweight score head estimates an "objectness" score for every encoder output position; the top-K positions are selected and used to generate initial anchor boxes as the positional queries for the decoder, while the content queries remain learnable embeddings. This gives the decoder a reasonable spatial initialisation from the very first layer, significantly accelerating convergence.

Look Forward Twice Box Refinement

Problem: When the decoder refines anchor boxes layer by layer, earlier layers receive only their own auxiliary loss gradients and minor loss gradient from the final output. Later layers see more accurate boxes but cannot propagate their refinement information back to help earlier layers, creating an optimisation imbalance.

Approach: Look Forward Twice maintains two forms of each layer's refined box: an undetached version for computing the current-layer loss (allowing gradients from later layers to flow back), and a detached version passed as input to the next layer. This lets later-layer losses additionally update earlier-layer parameters, accelerating convergence and providing richer update signals.

Contrastive DeNoising (CDN)

Problem: DETR-family models rely on Hungarian matching to assign queries to ground truths. Early in training, model outputs are random, causing matching assignments to fluctuate wildly between epochs — the same query may be assigned to object A one epoch and object B the next, cancelling out the learning signal.

Approach: During training, noisy versions of the ground-truth boxes (random translation/scaling + label flipping) are injected as extra queries and isolated from normal queries via attention masks. They are split into "positive" samples (supervising reconstruction of nearby GT) and "negative" samples (supervising rejection of distant spurious boxes). This provides strong, stable supervision without Hungarian matching and significantly stabilises and accelerates convergence.

Prediction Heads and Auxiliary Losses

Problem: If only the final decoder layer is supervised, intermediate layers receive no direct training signal; deeper networks also suffer from vanishing gradients in earlier layers.

Approach: Every decoder layer has its own independent classification head and bounding-box head, each contributing an auxiliary loss. Additionally, the Top-K encoder candidates compute a separate auxiliary

classification and box loss. This ensures the backbone, encoder, and all decoder layers are directly optimised.

Component	Configuration
Backbone	ResNet-50 (ImageNet pre-trained), all layers trainable
Multi-scale levels	3-scale feature pyramid
Encoder / Decoder	3 / 3 layers, hidden size 256, nhead 8, FFN 2048
Attention	Multi-scale deformable attention (pure PyTorch)
Query selection	Mixed (position from encoder Top-K, content learnable)
Number of queries	10
Box refinement	Look Forward Twice (independent bbox MLP per layer)
DeNoising	Contrastive DeNoising (positive/negative noisy queries)
Total parameters	~41.4 M

Table 1: DINO model architecture summary.

2.2 Training Strategy

Differential Learning Rates

Problem: The backbone is ImageNet pre-trained and already possesses general visual features, while the Transformer and prediction heads are trained from scratch. Using a single learning rate risks destroying the pre-trained backbone knowledge if it is too large, or making the Transformer converge too slowly if it is too small.

Approach: The backbone and remaining parameters are placed in separate parameter groups. The backbone learning rate is set to 1/10 of the rest (1e-5 vs 1e-4), preserving pre-trained knowledge while allowing new components to learn faster.

Loss Functions

Problem: In detection, the vast majority of queries correspond to background, causing severe class imbalance. Vanilla DETR uses softmax cross-entropy with a manually tuned no_object_weight, which requires careful adjustment and remains sensitive to extreme imbalance.

Approach: Classification uses sigmoid focal loss (α=0.25, γ=2.0), which adaptively down-weights easy negative samples without requiring a hand-crafted background weight. Bounding boxes use both L1 and GIoU losses (weights 5.0 and 2.0): L1 provides stable coordinate gradients, and GIoU supplies an overall box-alignment signal including the non-overlapping case. The Hungarian matching cost is also focal-loss-based to stay consistent with the training loss.

Optimizer and Scheduler

Problem: Transformer models are sensitive to weight decay and gradient scale; large update steps can cause attention weights to diverge, while small steps prevent learning.

Approach: AdamW is used with CosineAnnealingLR for smooth learning-rate decay over epochs. Gradient clipping (max_norm=0.1) guards against occasional gradient spikes that could destabilise parameters.

2.3 Data Augmentation

Problem: Although the street-view digit dataset has ~30k images, scene variety (lighting, viewpoint, resolution, noise) is limited. Without augmentation, the model easily overfits the training distribution and fails to generalise.

Approach: Both spatial and pixel augmentations are applied independently so each can be evaluated separately. To prevent spatial augmentations from cropping digits to near-nothing, boxes with insufficient area ratio or overlap are automatically discarded during crop/translate operations.

Category	Operation	Main Purpose
Spatial	RandomExpand	Simulate "object appears smaller", increases small-object samples
Spatial	RandomCrop	Simulate "only part of the scene is visible"
Spatial	RandomTranslation	Improve position invariance
Pixel	ColorJitter	Simulate lighting and colour-tone variation
Pixel	GaussianBlur	Simulate lens defocus
Pixel	IsoNoise (additive Gaussian)	Simulate sensor noise

Table 2: Training data augmentation. Spatial augmentations update bboxes accordingly.

2.4 Hyperparameter Summary

Hyperparameter	Value
Optimizer	AdamW
Backbone LR / Other LR	1e-5 / 1e-4
Weight decay	1e-4
LR scheduler	CosineAnnealingLR
Batch size	8
Gradient clip (max_norm)	0.1
Epochs (cumulative, multi-stage resume)	~250
Image size	min_side=600, max_side=900
Number of queries	10
Multi-scale levels	3
CDN groups	3
λ_cls / λ_bbox / λ_giou	1.0 / 5.0 / 2.0
Focal α / γ	0.25 / 2.0
Mixed precision	AMP enabled

Table 3: Final model hyperparameter settings.

3. Results

3.1 Best Model Performance

Metric	Value
Best validation mAP (COCO, IoU=0.5:0.95)	0.4611
Trainable parameters	~41.4 M
Test set images predicted	13,068

Table 4: Best model results summary.

3.2 Training Curves

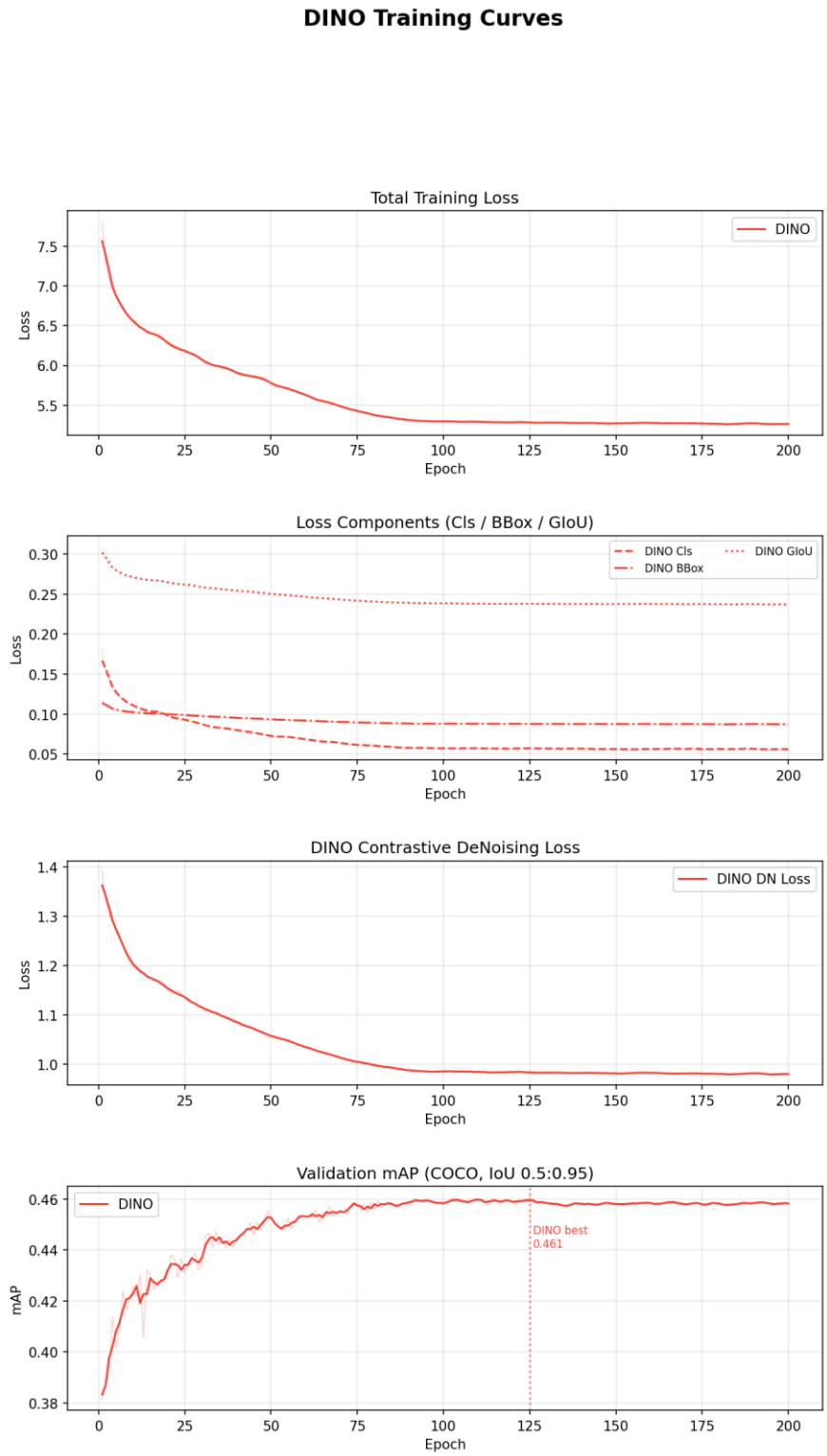

Figure 1: Training loss and validation mAP curves of the best model.

My best model was trained using the weights of the previous top-performing iteration as the initial weights, which allowed the validation mAP to start at a high baseline of 0.38. As observed from the training curves, the model exhibited a stable improvement in classification, bounding box localization, and shape refinement during the initial phase; however, by approximately the 20th epoch, the bounding box

coordinates had largely stabilized and the loss plateaued, while classification loss began to decline much more slowly. During the later stages of training, optimization primarily focused on refining the GIoU—specifically improving the shape alignment and overlap between the predicted boxes and ground truths. Although convergence slowed down and the validation mAP experienced some fluctuations in the final epochs, the diversity provided by the extensive data augmentation allowed the model to maintain a slight upward trajectory, eventually reaching a final mAP of 0.461.

3.3 Confusion Matrix

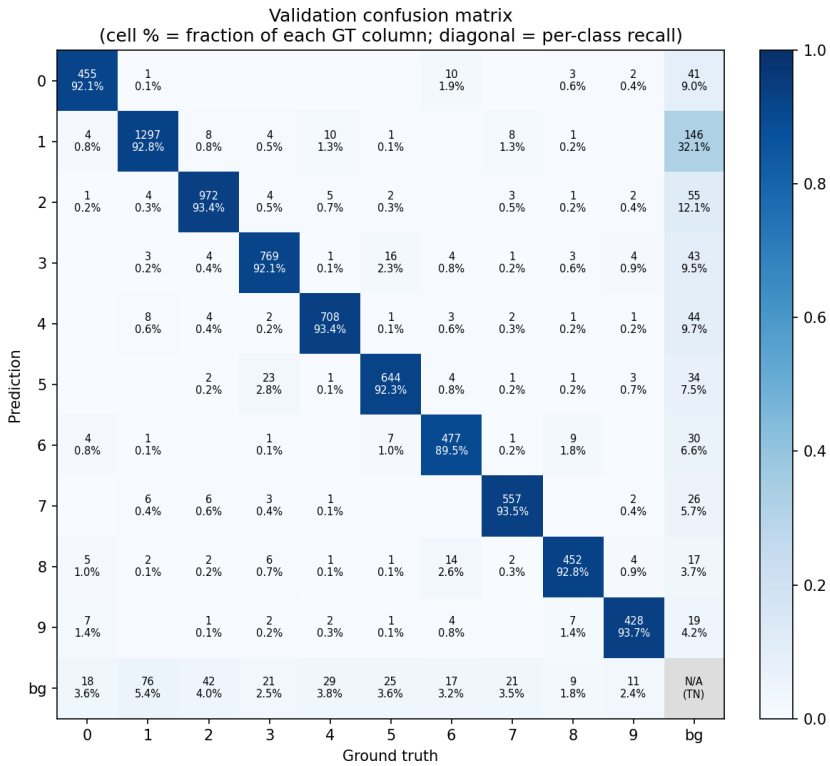

Figure 2: Confusion matrix of digit class 0-9 and the background. The percentage means the proportion of predicted class for one ground truth class.

The confusion matrix calculates the proportion of actual digits (ground truth) being classified as specific digits. Since background regions are not explicitly boxed during inference, they are represented as NA in the ground truth rows. However, because the background can be misclassified as a digit, the final column contains values across various digit rows.

Analysis of the 11-class confusion matrix reveals that most digits in the images are classified with high precision, with accuracies exceeding 92%. Specifically, the digit 6 is more frequently misclassified as 8 or 0, while the digit 5 is often confused with 3. This indicates that digits with similar visual structures are more prone to classification errors. For instance, 5 and 3 share nearly identical horizontal mirror-image characteristics; although horizontal flipping was not utilized as a data augmentation technique, their shared local features still lead to confusion. Regarding the background, a significant portion is misclassified as the digit 1. This is likely because the shape of 1 is a simple vertical line with minimal distinct features, causing linear textures or stripes in the background to be easily mistaken for this digit.

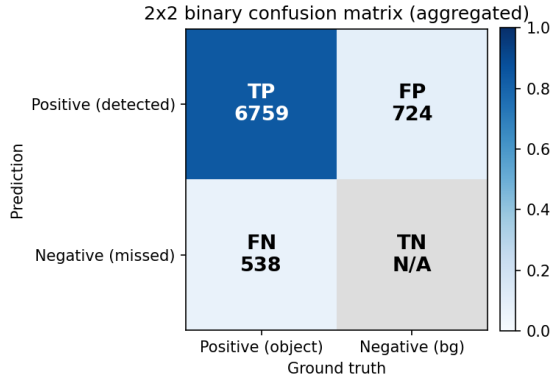

Figure 3: Aggregated binary confusion matrix of the positive objects and the background. The numbers mean the absolute count of predicted instances for each ground truth category.

This figure illustrates a 2x2 confusion matrix, where TP (True Positive) indicates correctly classified digits, FP (False Positive) denotes background regions misclassified as digits, and FN (False Negative) represents missed digits that were incorrectly treated as background.

The matrix reveals that while the vast majority of digits are successfully classified, a significant amount of background noise is erroneously detected as digits. Additionally, the proportion of actual digits missed and treated as background is slightly lower, though it still accounts for a noticeable fraction. To further improve the model, priority should be given to resolving the false positive issue—specifically, mitigating the misclassification of background regions as digits.

3.4 Cumulative Improvement

Starting from the first fully converged baseline, design decisions were added incrementally. The table below shows validation mAP at key milestones:

Stage	val mAP	Δ	Main Change
Tuned Baseline DINO	0.4241	—	DINO at resolution 480 with color jitter augmentation
+ Higher resolution	0.4386	+0.0145	Increased resolution to max-side 900 pixels
+ Full spatial + pixel augmentation	0.4611	+0.0225	Added Gaussian Blur, ISO noise, Translation, Random Crop, Random Expand

Table 5: Cumulative impact of major design decisions on validation mAP.

Table 5 shows that the largest gains come from data augmentation and higher resolution. Further architectural tuning on top of the full DINO baseline yielded limited returns, plateauing around 0.42 mAP. This highlights that once the model architecture is appropriate, remaining mAP improvements depend primarily on data diversity and thorough optimisation — data augmentation directly addresses both by varying the effective training distribution each epoch.

4. Additional Experiments

This section summarises ablation experiments where a single factor is changed relative to the best configuration. For each experiment, (a) the motivation for trying the change, (b) why it might not work, and (c) the actual result and analysis are provided.

4.1 Multi-Scale Levels: 3 vs 4

Problem: The DINO paper uses a 4-scale feature pyramid and reports +5.6 AP over 3 scales on COCO; intuitively, adding a higher-resolution scale should improve small-object detection.

Approach: A controlled experiment comparing 3 vs 4 scales was conducted — if the additional highest-resolution scale provides no tangible benefit on this task, using fewer scales is preferable to save VRAM and computation.

Multi-scale levels	val mAP
3	0.4611
4	0.4555

Table 6: Controlled experiment on number of multi-scale levels.

Result & Analysis: With all other factors fixed, 3 scales outperforms 4 scales by +0.0056 mAP while also reducing VRAM and training time. This demonstrates that "multi-scale benefit is task-dependent" — the +5.6 AP reported in the paper does not automatically transfer to all datasets. For digit detection, using too many scales is overkill; 3 scales is adopted.

4.2 Adaptive Feature Fusion (AFF)

Problem: A common issue with multi-scale features is that object features at higher backbone levels are diluted by background features. AFF (Huang & Wang, 2024) adds a learned spatial attention to each scale level, theoretically enabling the model to dynamically weight each level and prevent important features from being suppressed.

Approach: If AFF truly addresses feature dilution, mAP should increase after adding it. However, if the backbone and encoder already provide adequate features, incorrectly learned attention weights could suppress useful signals. A controlled on/off experiment was run to determine which effect dominates.

Use AFF	val mAP
Yes	0.4209
No	0.4386

Table 7: AFF on vs off (all other settings identical).

Result & Analysis: This experiment uses an intermediate model rather than the final one. Enabling AFF stalls at ~0.42, noticeably below 0.4386 without it. In this task (digits are relatively clear; background complexity is limited), the AFF mask appears to suppress features that are already sufficiently useful. AFF is not adopted in the final model.

4.3 Number of CDN Groups

Problem: More CDN groups provide denser denoising supervision per epoch; the DINO paper uses hundreds of groups. However, the number of CDN queries grows linearly with "groups × max GT per image", so too many groups slow training and dilute the learning signal for normal matching queries.

Approach: Since each image in this task typically contains fewer than 10 objects, many denoising groups should be unnecessary. This intuition was verified by comparing different group counts under controlled conditions.

CDN groups	val mAP

3	0.4611
10	0.4584

Table 8: Controlled experiment on number of CDN groups.

Result & Analysis: 3 and 10 groups produce nearly identical mAP (difference < 0.003), confirming that a small number of denoising groups already provides sufficient stable supervision in low-density detection scenarios. Increasing groups further only wastes memory. 3 groups is adopted.

4.4 Number of Object Queries

Problem: The query count must be ≥ the maximum number of objects per image, otherwise missed detections are inevitable. DINO defaults to 900 queries, designed for dense COCO objects. But this task has at most 6 digits per image (average 2.5) — using 900 queries means most are permanently matched to background, which can hurt learning.

Approach: Starting from the hypothesis that query count should match dataset object density, several smaller query counts were compared.

Number of queries	val mAP
10	0.4611
50	0.4536

Table 9: Controlled experiment on number of object queries.

Result & Analysis: Increasing queries from 10 to 50 drops mAP by ~0.008, confirming the hypothesis that excess queries dilute the learning signal. 10 queries is adopted.

4.5 Full Spatial + Pixel Augmentation

Problem: Although the street-view digit dataset is large, scene diversity is limited. Without augmentation, the model easily overfits the training set and fails to generalise. Object detection also requires explicit invariance to both position and appearance changes.

Approach: Spatial (RandomCrop / RandomExpand / RandomTranslation) and pixel (ColorJitter / GaussianBlur / IsoNoise) augmentations are applied together. Strong spatial augmentation carries risk — cropping away digits or translating objects out of frame introduces noise. To mitigate this, boxes with insufficient min_overlap or min_area_ratio are automatically discarded.

Augmentation	val mAP
ColorJitter only	0.4386
+ Full spatial + pixel augmentation	0.4611

Table 10: Cumulative effect of augmentation strategies.

Result & Analysis: Full augmentation brings +0.0225 mAP, the largest gain of any single change. Although a full ablation was not completed, RandomExpand is believed to be the single most effective operation, as it synthesises small-object samples and directly addresses the model's weak small-digit detection.

4.6 Failed Variants: Vanilla DETR and MI-DETR

Problem: Before committing to DINO, vanilla DETR (single-scale, softmax CE, no DN) and an MI-DETR variant (multi-interval queries + U-Net skip connections + extra auxiliary losses) were attempted, hoping simpler architectures could achieve comparable performance. Both suffered from slow convergence, difficulty in hyperparameter tuning, and a noticeably lower mAP ceiling.

Approach: The main development effort was fully redirected to DINO. Overall, DINO's core gains stem from multi-scale features (handling digits of varying sizes), deformable attention (computational efficiency), CDN (training stability), and mixed query selection (faster convergence). The final model clearly outperforms both vanilla DETR and MI-DETR.

5. References

[1] Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In European Conference on Computer Vision (ECCV), pp. 213–229.

[2] Zhang, H., Li, F., Liu, S., Zhang, L., Su, H., Zhu, J., Ni, L. M., & Shum, H.-Y. (2023). DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In International Conference on Learning Representations (ICLR). arXiv:2203.03605.

[3] Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable transformers for end-to-end object detection. In International Conference on Learning Representations (ICLR).

[4] Li, F., Zhang, H., Liu, S., Guo, J., Ni, L. M., & Zhang, L. (2022). DN-DETR: Accelerate DETR training by introducing query DeNoising. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

[5] Liu, S., Li, F., Zhang, H., Yang, X., Qi, X., Su, H., Zhu, J., & Zhang, L. (2022). DAB-DETR: Dynamic anchor boxes are better queries for DETR. In International Conference on Learning Representations (ICLR).

[6] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778.

[7] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2980–2988.

[8] Rezatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., & Savarese, S. (2019). Generalized intersection over union: A metric and a loss for bounding box regression. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 658–666.

[9] Wu, Y., & He, K. (2018). Group normalization. In European Conference on Computer Vision (ECCV), pp. 3–19.

[10] Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In International Conference on Learning Representations (ICLR).

[11] Loshchilov, I., & Hutter, F. (2017). SGDR: Stochastic gradient descent with warm restarts. In International Conference on Learning Representations (ICLR).

[12] Huang, Z., & Wang, J. (2024). Small object detection by DETR via information augmentation and adaptive feature fusion. arXiv:2401.08017.

[13] Solovyev, R., Wang, W., & Gabruseva, T. (2021). Weighted boxes fusion: Ensembling boxes from different object detection models. Image and Vision Computing, 107, 104117.

[14] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In European Conference on Computer Vision (ECCV), pp. 740–755.

[15] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In Advances in Neural Information Processing Systems (NeurIPS), Vol. 32.

[16] Official DINO repository: https://github.com/IDEA-Research/DINO